

Glossary: Gmail Quote Selected Extension

A reference of the concepts, APIs, and terms used to build this extension — from Chrome extension fundamentals to the InboxSDK integration and the specific bugs it fixes.

Chrome Extension Fundamentals

Manifest V3 (MV3) The current Chrome extension platform version, declared via `"manifest_version": 3` in `manifest.json`. Replaces the older background-page model with service workers and tightens permission scoping. This extension is built entirely on MV3.

manifest.json The extension's configuration file. Declares its name, version, permissions, background service worker, keyboard commands, and content scripts. It is the single source of truth for what the browser injects and where.

Service Worker (`background.js`) The extension's background process. Unlike the old persistent background page, a service worker is event-driven and can be unloaded by Chrome when idle, then woken up again when an event (like a context menu click) fires. It has no direct access to the Gmail page's DOM — it can only communicate with content scripts via messaging or inject scripts on demand.

Content Script (`content.js`, `inboxsdk.js`) JavaScript that Chrome injects directly into a matching web page (here, `https://mail.google.com/*`). Content scripts can read and modify the page's DOM, but run in an **isolated world** — sharing the DOM with the page but not its JavaScript variables or functions.

Isolated World The sandboxed JavaScript execution context Chrome gives content scripts. This is why `content.js` and `inboxsdk.js` can manipulate Gmail's DOM freely, but cannot directly call any of Gmail's own internal JavaScript functions — everything has to go through DOM events, clicks, and standard browser APIs.

Permissions & `host_permissions` Declared capabilities the extension requests: `contextMenus` (custom right-click menu items), `scripting` (inject code/files into tabs on demand), `activeTab` (temporary access to the currently focused tab). `host_permissions` scopes all of this to `https://mail.google.com/*` only.

Extension Trigger Mechanisms

Context Menu (`chrome.contextMenus`) Registers “Reply to all with Quote” as a right-click option that appears only when text is selected (`contexts: [ "selection" ]`). Created once on `chrome.runtime.onInstalled`.

Keyboard Command (`chrome.commands`) Declared in `manifest.json` under `commands.run-quote-reply`, bound to `Alt+Q` (Windows/Linux) or `MacCtrl+Q` (Mac). Fired via `chrome.commands.onCommand`.

`chrome.scripting.executeScript` Used two different ways in this extension:

1. To run an inline function `((() => window.getSelection().toString()))` in the active tab and retrieve its return value — this is how the keyboard shortcut path grabs the selected text, since the background service worker can’t read the page’s `window.getSelection()` itself.
2. As a **recovery mechanism**: force-injecting `inboxsdk.js` and `content.js` into a tab that doesn’t already have them running (see “Connection Recovery” below).

`chrome.tabs.sendMessage` / `chrome.runtime.onMessage` The message-passing bridge between the background service worker and the content script. The background sends `{ action: "triggerReply", text }`; the content script listens for it and acknowledges receipt via `sendResponse`.

InboxSDK

InboxSDK A third-party JavaScript library (`inboxsdk.js`, bundled locally, ~2.2MB) purpose-built for writing Gmail browser extensions. It abstracts away Gmail’s unstable internal DOM structure and gives extensions a stable, documented API for hooking into Gmail’s UI — compose windows, threads, toolbars, etc.

`InboxSDK.load(version, appId)` The SDK’s entry point. Called once per page load in `content.js`. Returns a Promise that resolves with an `sdk` object once the SDK has attached itself to Gmail’s interface. `version` is the SDK’s API version (2); `appId` is a free identifier registered at `inboxsdk.com`, used for the SDK’s own usage tracking.

`ComposeView` InboxSDK’s abstraction over a single Gmail compose or reply window. Instead of an extension having to locate and interpret raw `<div contenteditable>` elements, InboxSDK hands back a `ComposeView` object with a stable API.

`sdk.Compose.registerComposeViewHandler(handler)` Registers a persistent callback that InboxSDK invokes once for every compose/reply view that exists or opens on the page, for as

long as the page is loaded — including ones that were already open at registration time. This replaces the need for a `MutationObserver` watching the DOM for new compose boxes.

`composeView.insertHTMLIntoBodyAtCursor(html)` The core InboxSDK method this extension relies on to insert the quoted text. It inserts an HTML string at the current cursor position inside the compose body, handling cursor placement and rich-text sanitization internally — replacing what used to be a fragile three-tier fallback chain (Clipboard API → `execCommand('insertHTML')` → raw `insertAdjacentHTML`).

`composeView.isMinimized()` Reports whether a given compose view is currently minimized (collapsed to a small bar at the bottom of Gmail). Used to skip minimized drafts when looking for “the” open compose view to insert into.

`composeView.on('destroy', callback)` InboxSDK’s event hook fired when a compose view is closed (sent, discarded, or otherwise removed). Used to prune closed views out of the extension’s tracked list so it never tries to insert into a stale, gone compose view.

Core Extension Logic

`pendingQuoteText` A module-level variable in `content.js` that temporarily holds the selected quote text between the moment the user triggers the extension and the moment a *newly opened* compose view becomes available. Cleared immediately after use to prevent the same quote being inserted twice.

`activeComposeViews` An array in `content.js` that tracks every compose/reply view currently open on the page. Populated inside `registerComposeViewHandler` and pruned on each view’s `destroy` event. This is what lets the extension distinguish “insert into the reply the user already has open” from “open a brand-new reply and insert once it appears.”

Reply Button Discovery When no compose view is already open, the extension must first get Gmail to open one, by clicking a real “Reply” or “Reply All” button on the page (InboxSDK does not open replies on the extension’s behalf). The discovery logic:

- `findNearestReplyButton` — walks up the DOM from the text-selection anchor through ancestor containers, looking for the closest visible reply control.
- `findReplyButton` — within a given container, filters to visible elements (`offsetWidth > 0 && offsetHeight > 0`) and prefers a “Reply All” match over a plain “Reply” match.
- `isReplyButton` / `isReplyAllButton` / `getHay` — text-matching helpers that check `innerText`, `aria-label`, and `data-tooltip` for the words “reply” / “all” in English, French (“répondre” / “tous”), and Spanish (“responder” / “todos”).
- **Fail-safe:** if no button is found anywhere, the extension dispatches a synthetic `r` keydown event, which is Gmail’s own native reply shortcut.

Blockquote Construction & Escaping The quoted text is wrapped in a `<blockquote>` with inline CSS (blue left border, italic text, light-blue background) before being handed to InboxSDK. `escapeHtml()` escapes `&`, `<`, `>`, and `"` in the user's selected text first, preventing any HTML/script the user selected from being executed when inserted into the compose body (XSS prevention).

Connection Recovery (`sendQuoteReply`) `chrome.tabs.sendMessage` fails with *"Could not establish connection. Receiving end does not exist"* when no content script listener exists in the target tab. This happens when:

1. The Gmail tab was already open **before** the extension was installed or reloaded — manifest-declared content scripts are only auto-injected on new page loads, not into already-open tabs.
2. A brief race condition right after a Gmail single-page-app navigation, before the content script's listener has finished registering.

`sendQuoteReply` catches this specific error, force re-injects `inboxsdk.js` + `content.js` via `chrome.scripting.executeScript`, waits briefly, and retries the message once before giving up.

Bugs Fixed During Development

Issue #1 — Insert into an already-open compose box Originally, triggering the extension always tried to click a reply button, even if the user had already manually opened a reply draft. Fixed by checking `activeComposeViews` first via `getOpenComposeView()`; if a non-minimized compose view already exists, the quote is inserted directly into it and the reply-button-click path is skipped entirely.

Issue #2 — Extra blank line after the quote The inserted HTML originally ended with `<blockquote>...</blockquote><br>`. Since `<blockquote>` is already a block-level element, the trailing `<br>` pushed the cursor down one extra blank line below the quote. Fixed by removing the trailing `<br>`, so the cursor now lands on the line immediately after the quoted block.

Superseded Techniques (Pre-InboxSDK)

These approaches were used in the original implementation and were replaced when the extension adopted InboxSDK. Documented here for historical context — see `doc/flow.md`'s Architecture Comparison table for the full before/after.

MutationObserver polling Watched `document.body` for DOM changes to detect when Gmail's compose box appeared, with a 5-second timeout safety net. Replaced by `registerComposeViewHandler`, which fires as an event rather than requiring polling.

Clipboard API fallback chain The original HTML insertion strategy tried, in order: the modern async Clipboard API (`navigator.clipboard.write + ClipboardItem + document.execCommand('paste')`), then `document.execCommand('insertHTML')`, then raw `insertAdjacentHTML` as a last resort. Replaced entirely by `insertHTMLIntoBodyAtCursor`.

Euclidean distance button scoring (`Math.hypot`) The original reply-button-finding algorithm computed the pixel distance between the text selection's bounding rectangle and every visible reply button on the page, weighting “Reply All” candidates with a scoring bonus. Replaced by the simpler ancestor-walk approach (`findNearestReplyButton`), which is cheaper and just as reliable in practice.